

ECE 276B Project 1: Dynamic Programming for Door-Key Navigation

Shih-Ting Chen

Abstract—This project studies an autonomous navigation problem in the MiniGrid Door-Key environment. The agent must reach a goal while minimizing a non-uniform action cost. Since all action costs are positive, I formulate the task as a deterministic shortest-path Markov Decision Process and solve it using dynamic programming over the full discrete state space.

I. INTRODUCTION

Door-key navigation is a simplified robotics planning problem. The robot must move through a grid map, interact with objects such as keys and locked doors, and reach a target location. This is related to mobile robot navigation, warehouse robotics, and task planning, where a robot must reason not only about motion but also about discrete object interactions. In this project, I solve the known-map and random-map settings using one general planner. The planner searches over the augmented state space containing the robot pose, key possession, and door status, and returns an optimal action sequence with minimum total cost.

II. PROBLEM STATEMENT

We formulate the Door-Key problem as a finite-horizon Markov Decision Process (MDP).

A. State Space

The state is defined as

$$x = (p_x, p_y, d, k, o), \quad (1)$$

where:

- $(p_x, p_y) \in \{0, \dots, W-1\} \times \{0, \dots, H-1\}$ is the agent position,
- $d \in \{0, 1, 2, 3\}$ represents the agent orientation (right, down, left, up),
- $k \in \{0, 1\}$ indicates whether the agent is carrying the key,
- $o \in \{0, 1\}$ indicates whether the door is open.

B. Control Space

The control space is

$$\mathcal{U} = \{\text{MF, TL, TR, PK, UD}\}, \quad (2)$$

corresponding to Move Forward, Turn Left, Turn Right, Pick Key, and Unlock Door.

C. Motion Model

The system evolves according to a deterministic transition function:

$$x_{t+1} = f(x_t, u_t), \quad (3)$$

where f encodes:

- Position updates when moving forward,
- Orientation updates when turning,
- Key acquisition when the agent executes PK at the key location,
- Door unlocking when executing UD at the door with a key.

D. Initial State

The initial state is defined as:

$$x_0 = (p_x^0, p_y^0, d^0, k^0, o^0), \quad (4)$$

where (p_x^0, p_y^0) and d^0 are provided by the environment configuration.

For all environments, the agent initially does not carry the key:

$$k^0 = 0. \quad (5)$$

The initial door state o^0 depends on the environment:

$$o^0 = \begin{cases} 1, & \text{if the door is initially open,} \\ 0, & \text{if the door is locked.} \end{cases} \quad (6)$$

E. Planning Horizon

We consider a finite-horizon planning problem with horizon T . The objective is to reach the goal in a finite number of steps while minimizing the cumulative cost:

$$V^\pi(x_0) = q(x_T) + \sum_{t=0}^{T-1} \ell(x_t, u_t). \quad (7)$$

Since the state space is finite and all stage costs are strictly positive, the optimal policy will reach the goal in a finite number of steps. Therefore, the planning horizon can be chosen sufficiently large:

$$T \geq T^*, \quad (8)$$

where T^* is the minimum number of steps required to reach the goal under the optimal policy.

F. Stage Cost

The stage cost function $\ell(x, u)$ is defined as:

$$\ell(x, u) = \begin{cases} 1, & u \in \{\text{TL}, \text{TR}\} \\ 3, & u = \text{MF} \\ 2, & u = \text{PK} \\ 5, & u = \text{UD} \end{cases} \quad (9)$$

G. Terminal Cost

We define a terminal cost function $q(x)$ associated with terminal states:

$$q(x) = \begin{cases} 0, & x \in \mathcal{X}_{\text{goal}}, \\ +\infty, & \text{otherwise.} \end{cases} \quad (10)$$

The terminal cost enforces that only trajectories that reach the goal state are considered valid solutions. Any trajectory that does not reach the goal incurs an infinite cost.

H. Objective Function

With the terminal cost, the total cost becomes:

$$V^\pi(x_0) = q(x_T) + \sum_{t=0}^{T-1} \ell(x_t, u_t). \quad (11)$$

subject to the system motion model, where π is a policy.

I. Terminal Condition

The episode terminates when the agent reaches the goal position:

$$(p_x, p_y) = (p_x^{\text{goal}}, p_y^{\text{goal}}). \quad (12)$$

J. Optimal Policy

The optimal policy is defined as:

$$\pi^*(x) = \arg \min_{u \in \mathcal{U}} [\ell(x, u) + V^*(f(x, u))], \quad (13)$$

where the optimal value function satisfies the Bellman equation:

$$V^*(x) = \min_{u \in \mathcal{U}} [\ell(x, u) + V^*(f(x, u))]. \quad (14)$$

III. TECHNICAL APPROACH

We solve the Door-Key problem using dynamic programming (DP) based on the Bellman optimality principle. The problem is formulated as a finite, deterministic Markov Decision Process (MDP), and we compute the optimal value function and policy over the entire state space.

A. Bellman Optimality Equation

Let $V^*(x)$ denote the optimal cost-to-go from state x . The Bellman optimality equation is:

$$V^*(x) = \min_{u \in \mathcal{U}} [\ell(x, u) + V^*(f(x, u))]. \quad (15)$$

For terminal states (goal states), we define:

$$V^*(x) = 0, \quad x \in \mathcal{X}_{\text{goal}}. \quad (16)$$

For all other states, the value function is computed recursively.

B. Value Iteration

We compute the optimal value function using value iteration. Let $V_k(x)$ denote the value function at iteration k . The update rule is:

$$V_{k+1}(x) = \min_{u \in \mathcal{U}} [\ell(x, u) + V_k(f(x, u))]. \quad (17)$$

The algorithm is initialized as:

$$V_0(x) = \begin{cases} 0, & x \in \mathcal{X}_{\text{goal}} \\ +\infty, & \text{otherwise} \end{cases} \quad (18)$$

The iteration continues until convergence:

$$\max_x |V_{k+1}(x) - V_k(x)| < \epsilon. \quad (19)$$

Since the state space is finite and all costs are nonnegative, value iteration is guaranteed to converge to the optimal value function.

C. Policy Extraction

Once the optimal value function $V^*(x)$ is obtained, the optimal policy is derived as:

$$\pi^*(x) = \arg \min_{u \in \mathcal{U}} [\ell(x, u) + V^*(f(x, u))]. \quad (20)$$

This defines a mapping:

$$\pi^* : \mathcal{X} \rightarrow \mathcal{U}. \quad (21)$$

At runtime, the agent selects actions according to:

$$u_t = \pi^*(x_t). \quad (22)$$

D. State Transition Model

The transition function $f(x, u)$ is deterministic and depends on the policy type.

a) *Turning Policies:*

$$d_{t+1} = \begin{cases} (d_t - 1) \bmod 4, & u_t = \text{TL}, \\ (d_t + 1) \bmod 4, & u_t = \text{TR}. \end{cases} \quad (23)$$

b) *Forward Motion:* Define the direction vector:

$$v(d) = \begin{cases} (1, 0), & d = 0, \\ (0, 1), & d = 1, \\ (-1, 0), & d = 2, \\ (0, -1), & d = 3. \end{cases} \quad (24)$$

The candidate position is:

$$p' = p + v(d). \quad (25)$$

If p' is traversable, then:

$$p_{t+1} = p'. \quad (26)$$

Otherwise:

$$p_{t+1} = p_t. \quad (27)$$

c) *Key Pickup:*

$$k_{t+1} = \begin{cases} 1, & u_t = \text{PK and } p_t = p_{\text{key}}, \\ k_t, & \text{otherwise.} \end{cases} \quad (28)$$

d) *Door Unlocking*:

$$o_{t+1} = \begin{cases} 1, & u_t = \text{UD}, k_t = 1, \text{ and door is in front,} \\ o_t, & \text{otherwise.} \end{cases} \quad (29)$$

E. Algorithm Description

Algorithm 1 Value Iteration for Door-Key Problem

```

1: Initialize  $V(x)$  for all states
2: repeat
3:   for each state  $x \in \mathcal{X}$  do
4:     if  $x$  is terminal then
5:        $V_{\text{new}}(x) = 0$ 
6:     else
7:        $V_{\text{new}}(x) = \min_u [\ell(x, u) + V(f(x, u))]$ 
8:     end if
9:   end for
10:  Update  $V(x) \leftarrow V_{\text{new}}(x)$ 
11: until convergence
12: Extract policy  $\pi^*(x)$  using Bellman equation

```

F. Application to Part (A) and Part (B)

For Part (A), the environment is fully known. We perform value iteration separately for each environment and extract the corresponding optimal policy sequence.

For Part (B), the environment belongs to a family of random maps. We apply the same DP procedure to compute the policy for each observed environment, ensuring that the method generalizes across all configurations.

G. Complexity Analysis

Let the number of states be:

$$|\mathcal{X}| = W \times H \times 4 \times 2 \times 2. \quad (30)$$

Each value iteration step requires evaluating all policies:

$$O(|\mathcal{X}| \cdot |\mathcal{U}|). \quad (31)$$

Thus, the total complexity is:

$$O(K \cdot |\mathcal{X}| \cdot |\mathcal{U}|), \quad (32)$$

where K is the number of iterations until convergence.

IV. RESULTS

In this section, we present the trajectory visualizations generated by our dynamic programming policy. For each environment, the figure shows the environment layout, start location, key, door, goal, and the trajectory followed by the agent. The total cost and number of steps are shown in the title of each figure.

In addition to the static trajectory figures shown in this report, animated GIFs are also generated for all environments to demonstrate the full execution sequence of the agent, including turning, moving, key pickup, door unlocking, and goal reaching.

A. Part (A): Known Map Results

For Part (A), we evaluate the planner on all seven known Door-Key environments. A separate optimal policy/action sequence is computed for each known map.

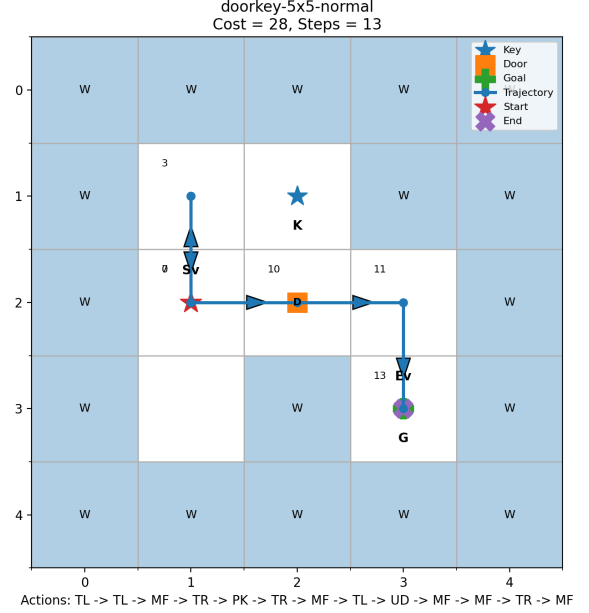


Fig. 1: Optimal trajectory for doorkey-5x5-normal.

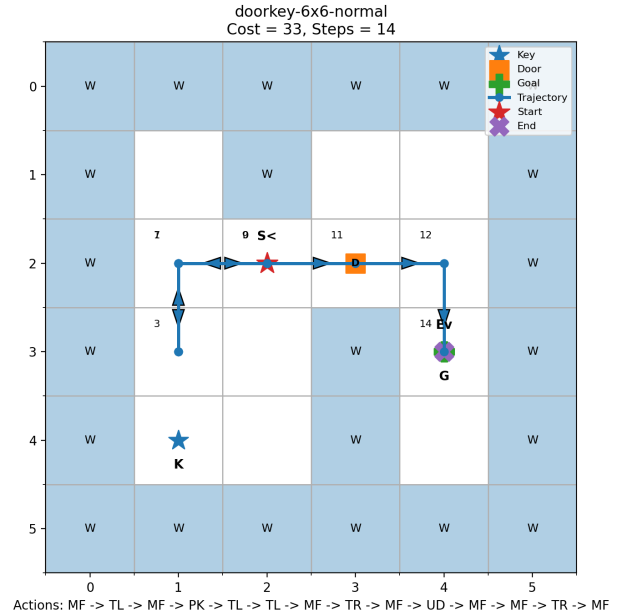


Fig. 2: Optimal trajectory for doorkey-6x6-normal.

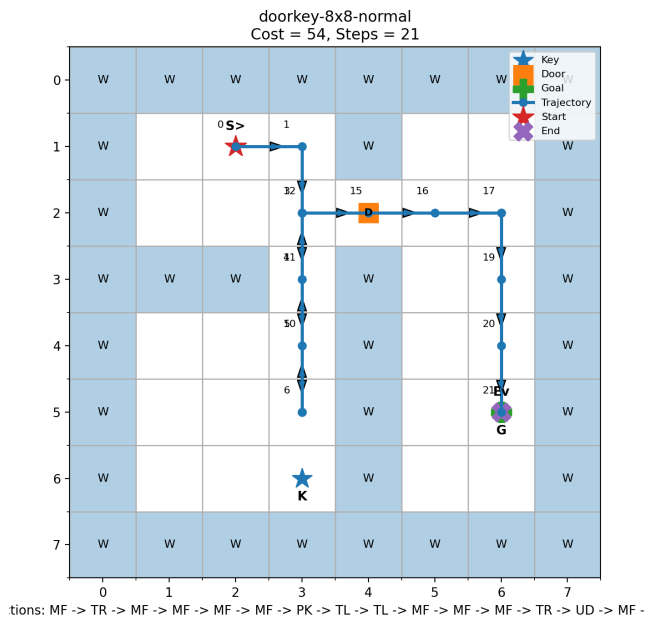


Fig. 3: Optimal trajectory for doorkey-8x8-normal.

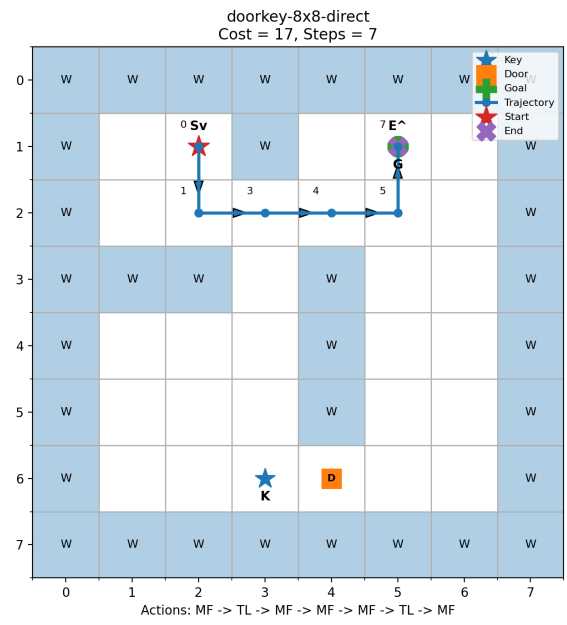


Fig. 5: Optimal trajectory for doorkey-8x8-direct.

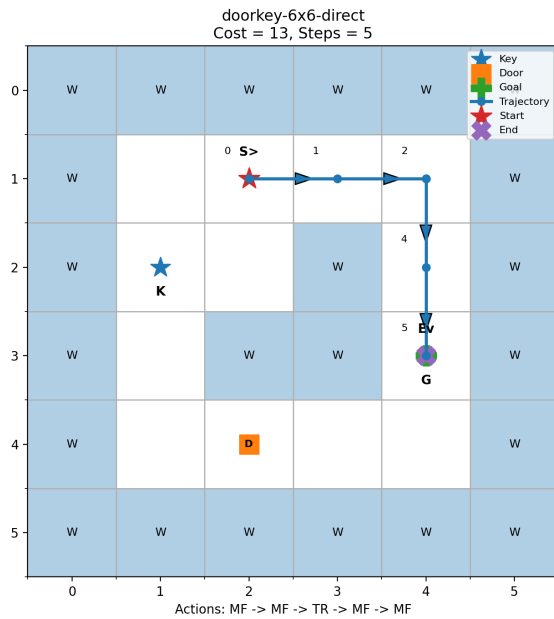


Fig. 4: Optimal trajectory for doorkey-6x6-direct.

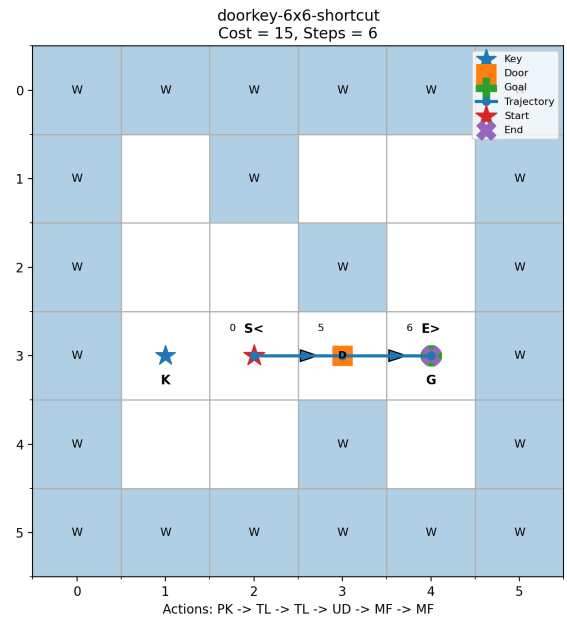


Fig. 6: Optimal trajectory for doorkey-6x6-shortcut.

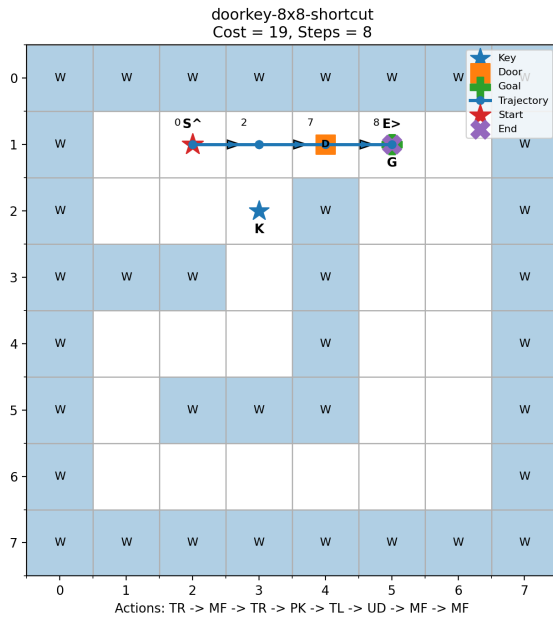


Fig. 7: Optimal trajectory for doorkey-8x8-shortcut.

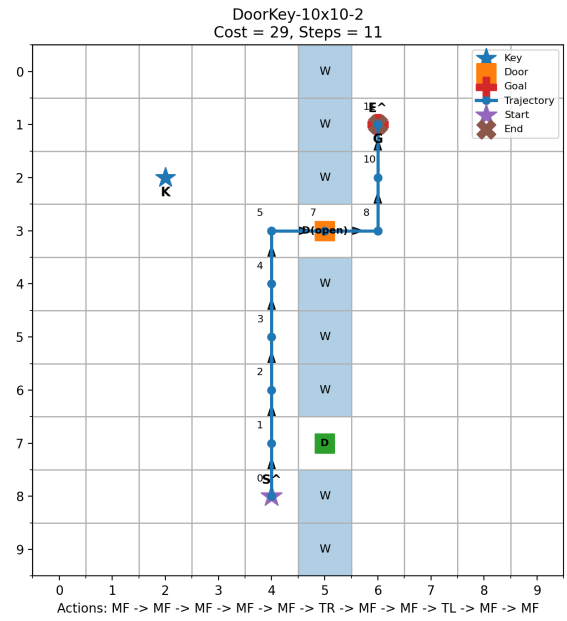


Fig. 9: Trajectory for random environment DoorKey-10x10-2.

B. Part (B): Random Map Results

For Part (B), we compute a single unified control policy and apply the same policy to all 36 random environments. The environments are ordered according to their file names, from DoorKey-10x10-1.env to DoorKey-10x10-36.env. No environment-specific replanning is performed during evaluation.

The following figures show the trajectory generated by the same policy for each random environment.

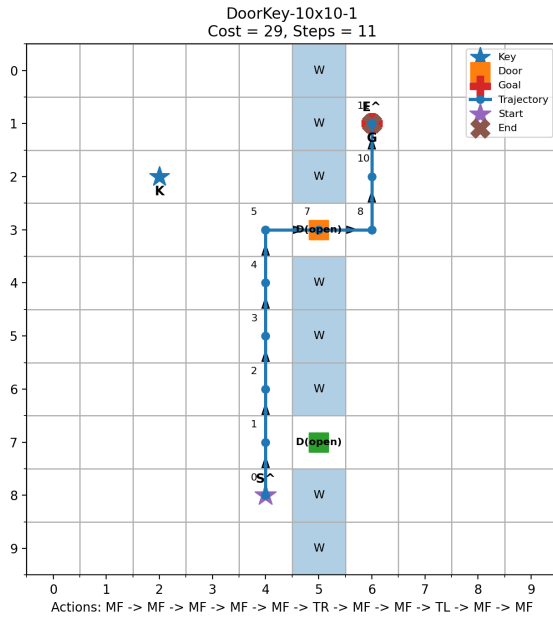


Fig. 8: Trajectory for random environment DoorKey-10x10-1.

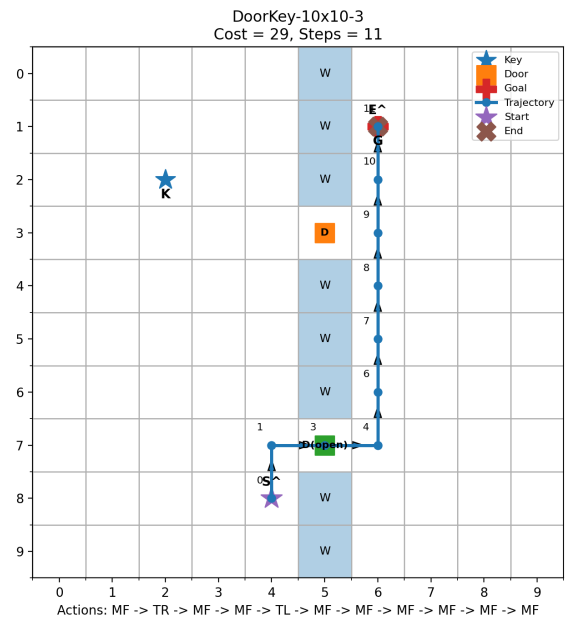


Fig. 10: Trajectory for random environment DoorKey-10x10-3.

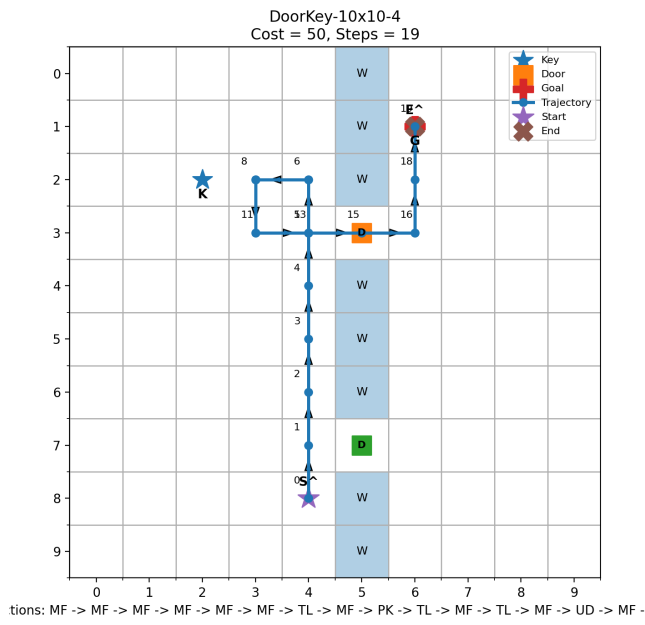


Fig. 11: Trajectory for random environment DoorKey-10x10-4.

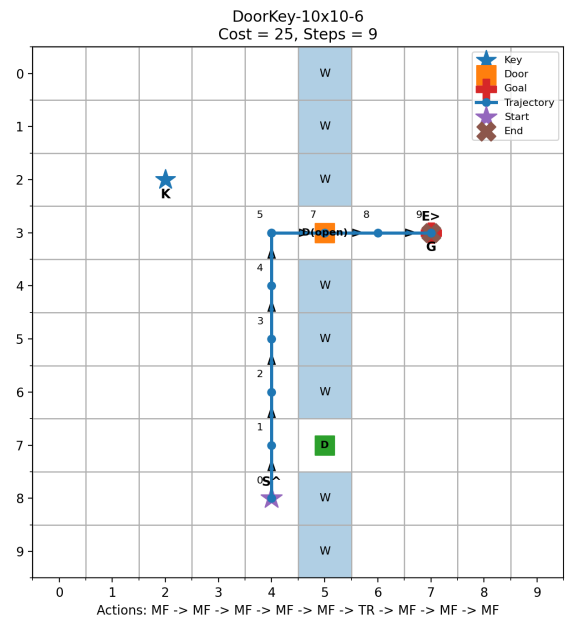


Fig. 13: Trajectory for random environment DoorKey-10x10-6.

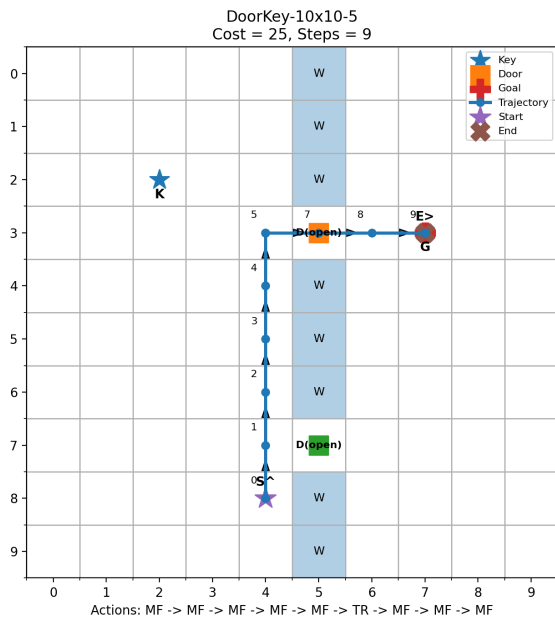


Fig. 12: Trajectory for random environment DoorKey-10x10-5.

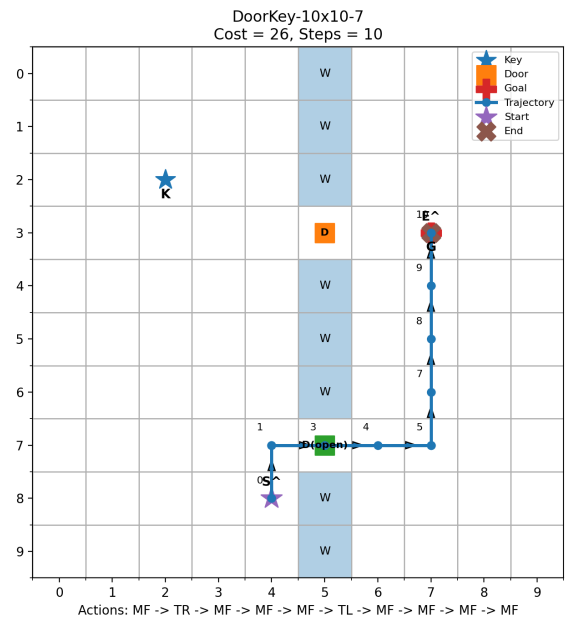


Fig. 14: Trajectory for random environment DoorKey-10x10-7.

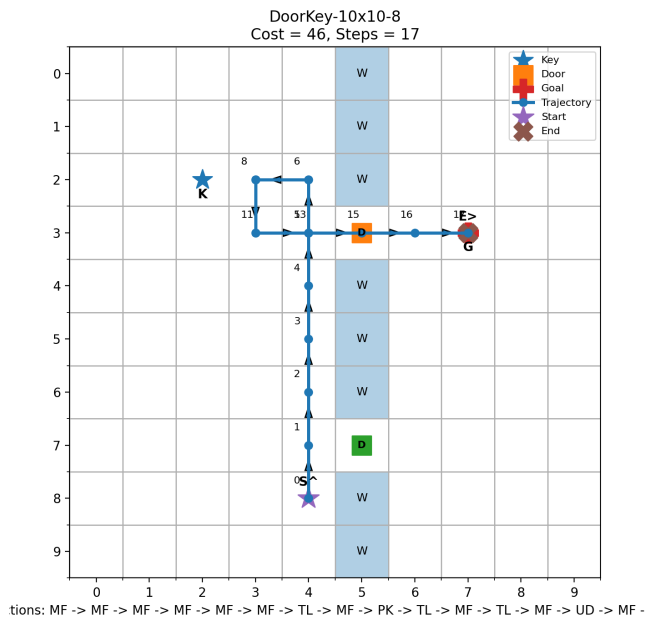


Fig. 15: Trajectory for random environment DoorKey-10x10-8.

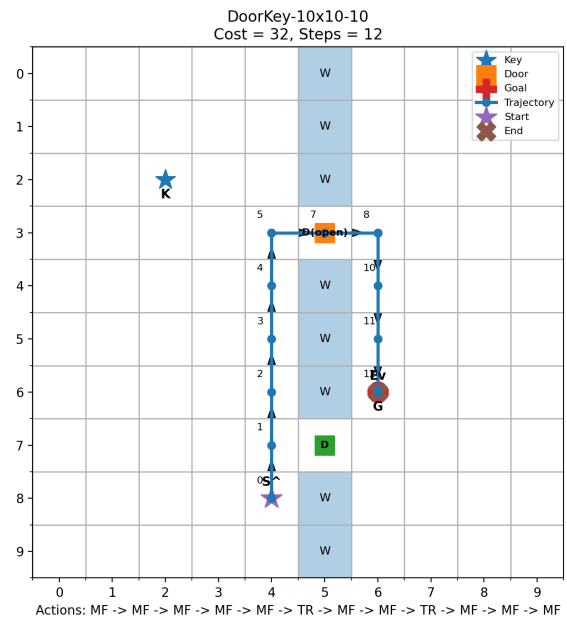


Fig. 17: Trajectory for random environment DoorKey-10x10-10.

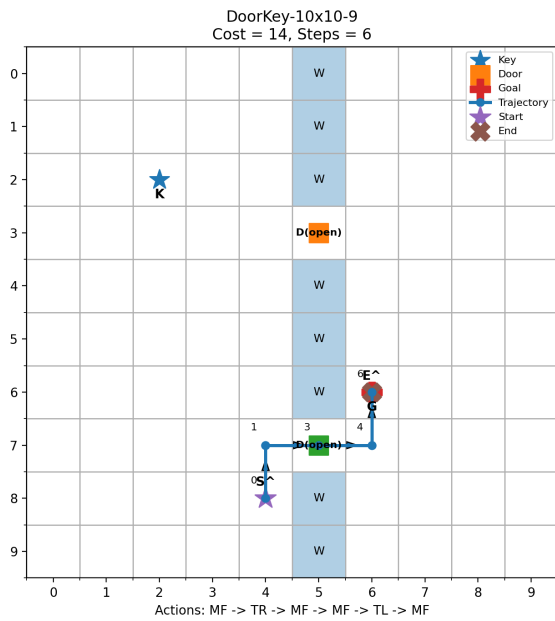


Fig. 16: Trajectory for random environment DoorKey-10x10-9.

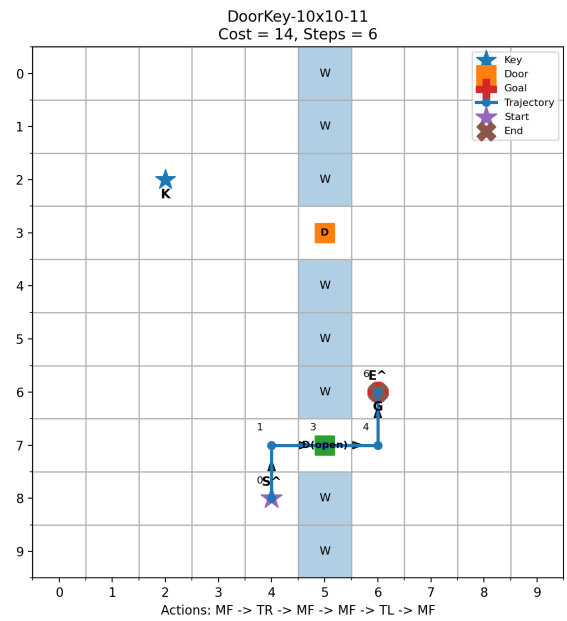


Fig. 18: Trajectory for random environment DoorKey-10x10-11.

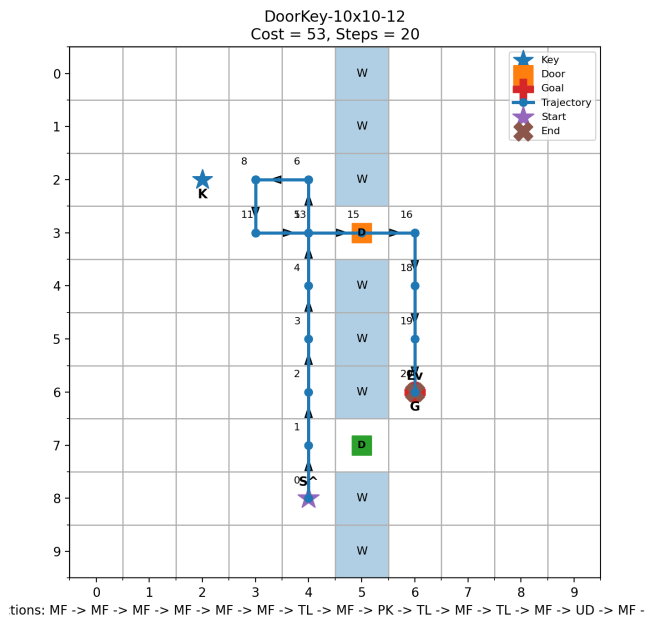


Fig. 19: Trajectory for random environment DoorKey-10x10-12.

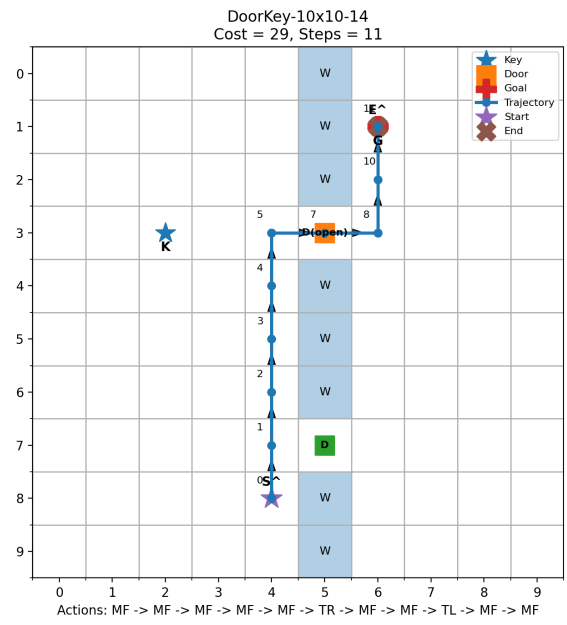


Fig. 21: Trajectory for random environment DoorKey-10x10-14.

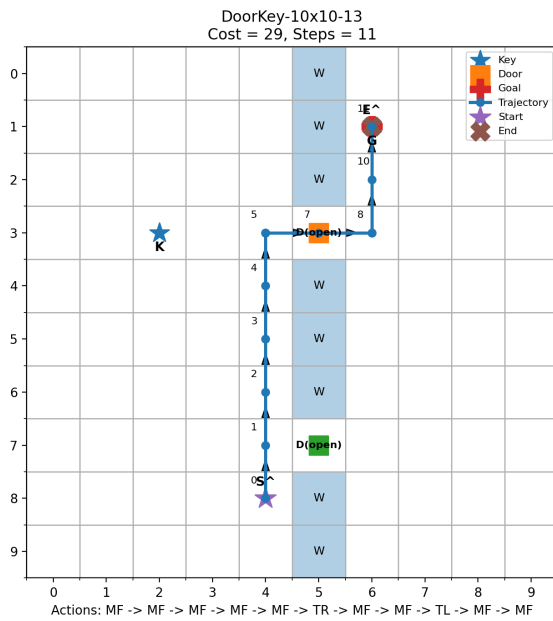


Fig. 20: Trajectory for random environment DoorKey-10x10-13.

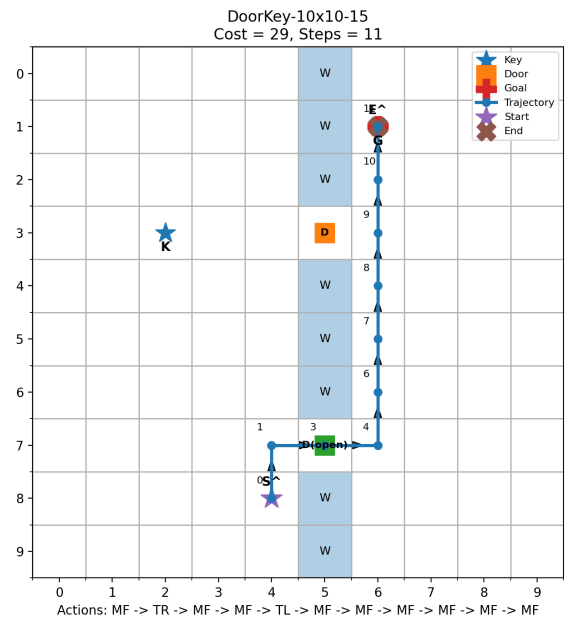


Fig. 22: Trajectory for random environment DoorKey-10x10-15.

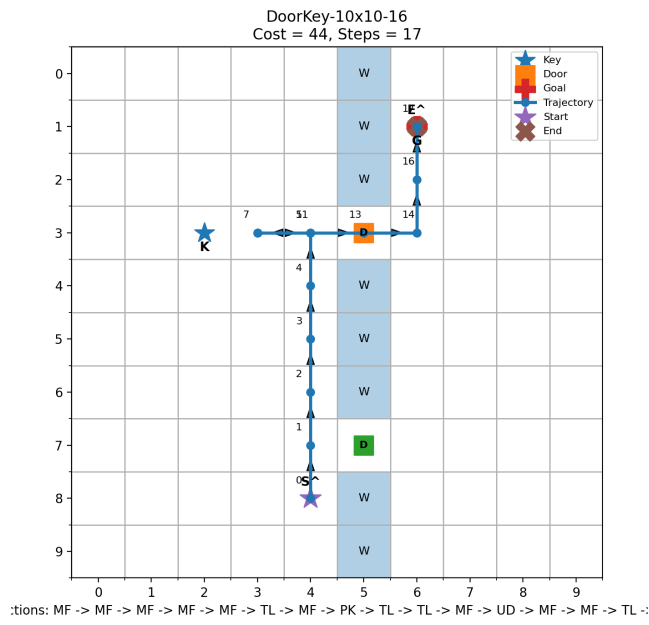


Fig. 23: Trajectory for random environment DoorKey-10x10-16.

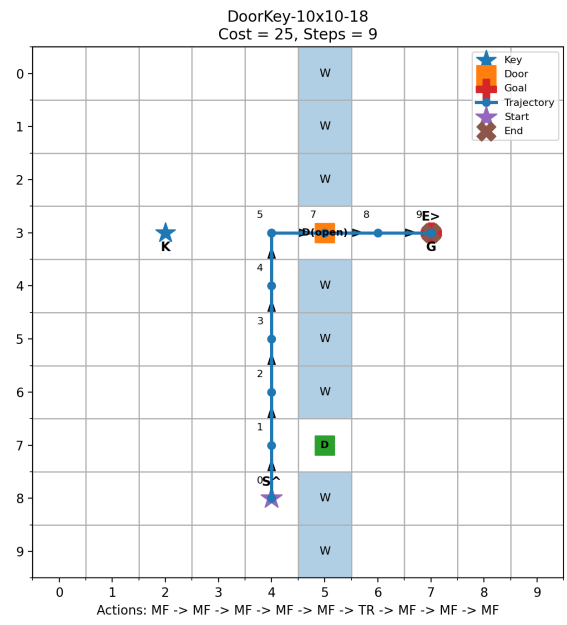


Fig. 25: Trajectory for random environment DoorKey-10x10-18.

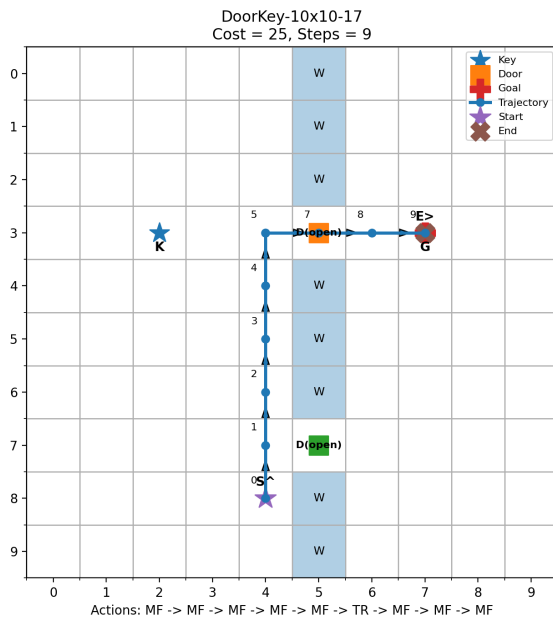


Fig. 24: Trajectory for random environment DoorKey-10x10-17.

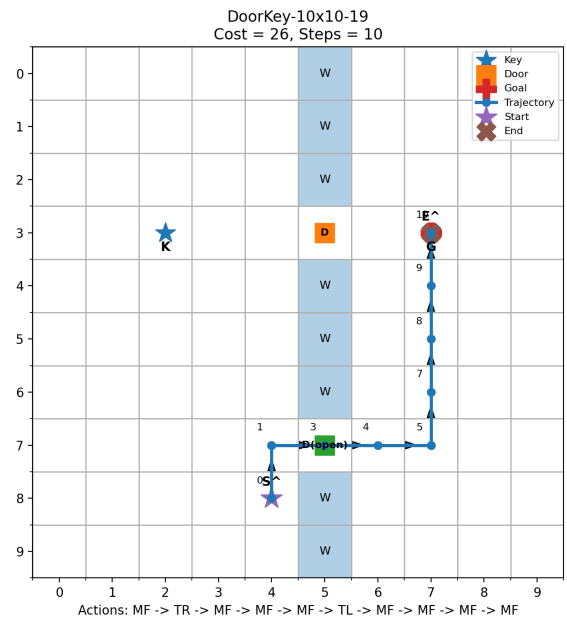


Fig. 26: Trajectory for random environment DoorKey-10x10-19.

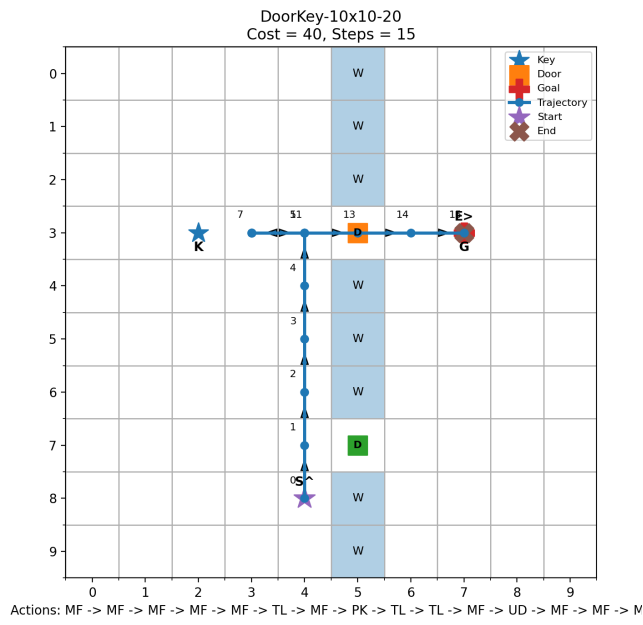


Fig. 27: Trajectory for random environment DoorKey-10x10-20.

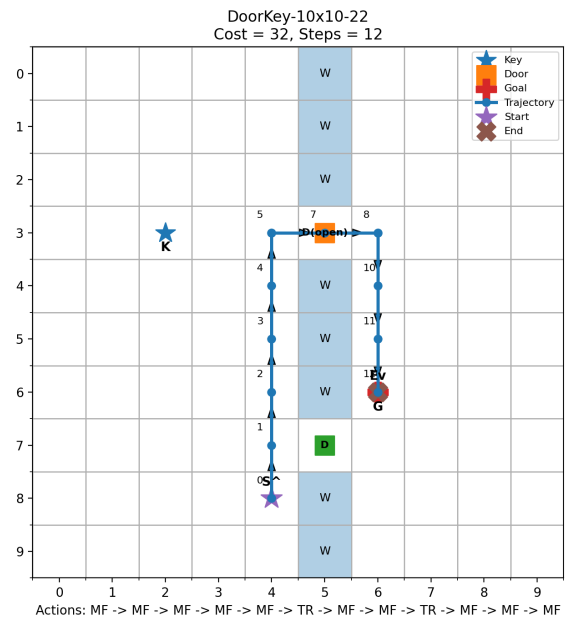


Fig. 29: Trajectory for random environment DoorKey-10x10-22.

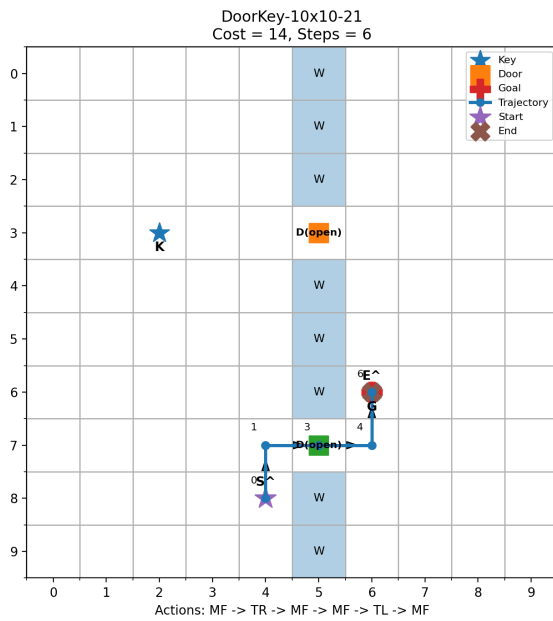


Fig. 28: Trajectory for random environment DoorKey-10x10-21.

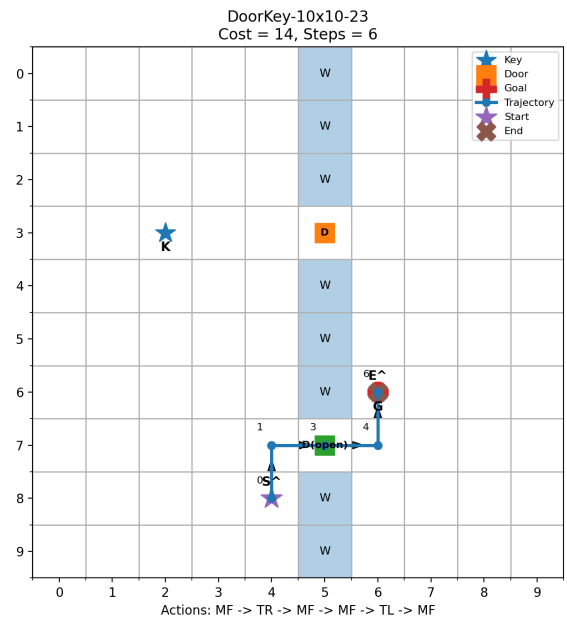


Fig. 30: Trajectory for random environment DoorKey-10x10-23.

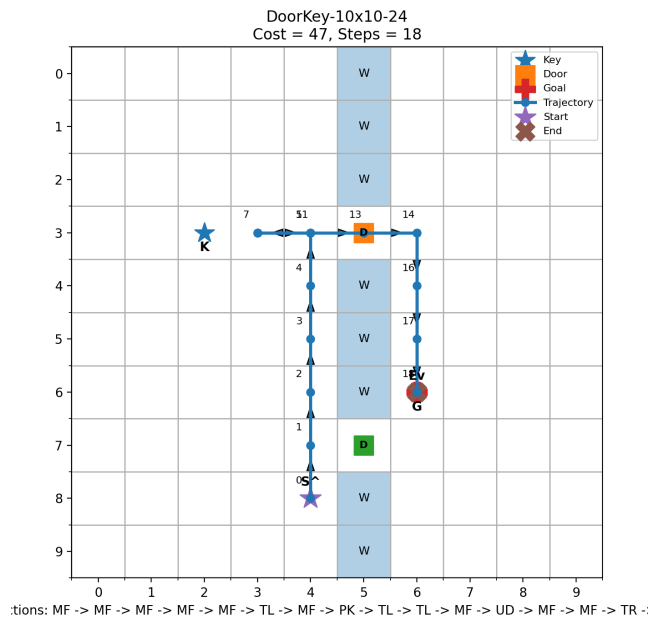


Fig. 31: Trajectory for random environment DoorKey-10x10-24.

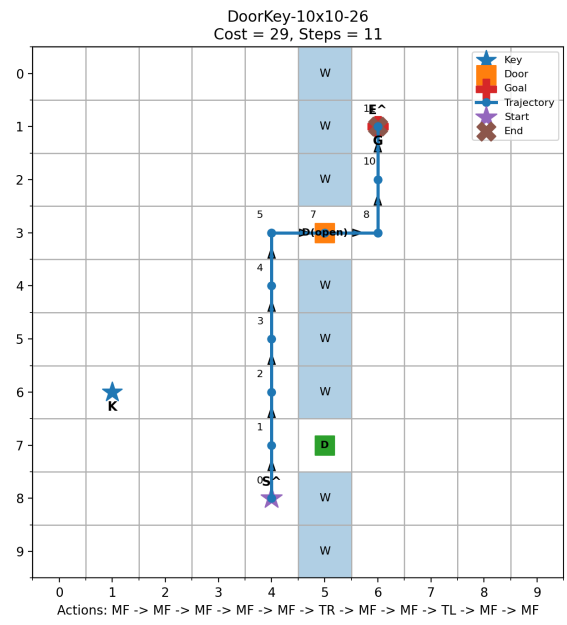


Fig. 33: Trajectory for random environment DoorKey-10x10-26.

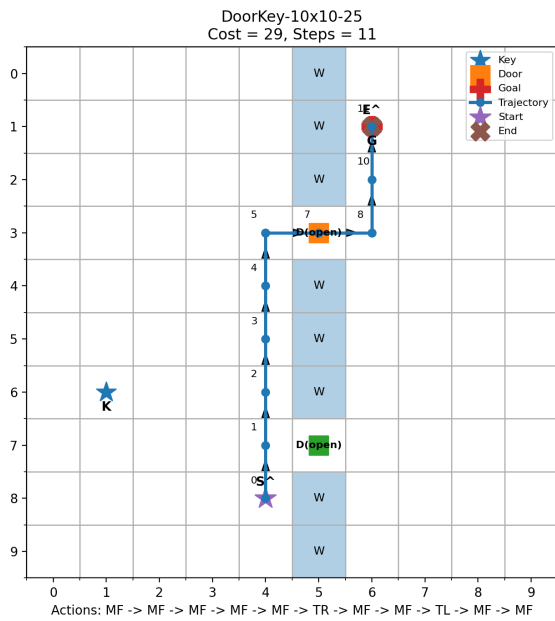


Fig. 32: Trajectory for random environment DoorKey-10x10-25.

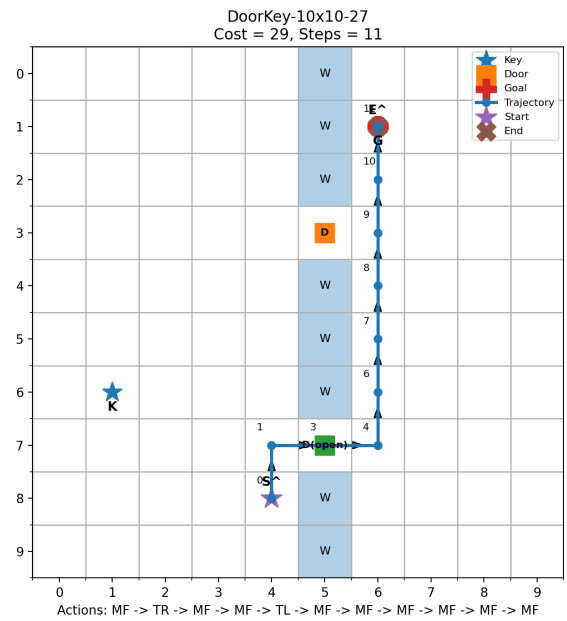
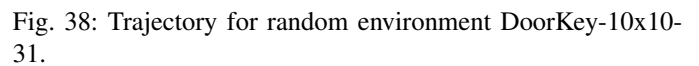
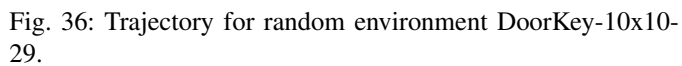
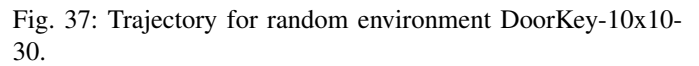
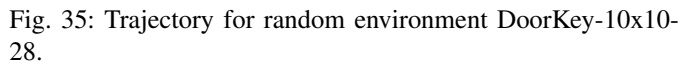


Fig. 34: Trajectory for random environment DoorKey-10x10-27.



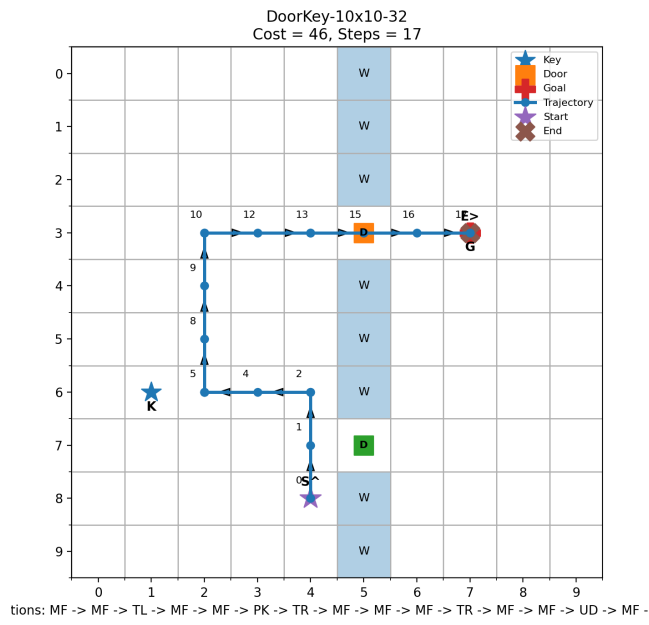


Fig. 39: Trajectory for random environment DoorKey-10x10-32.

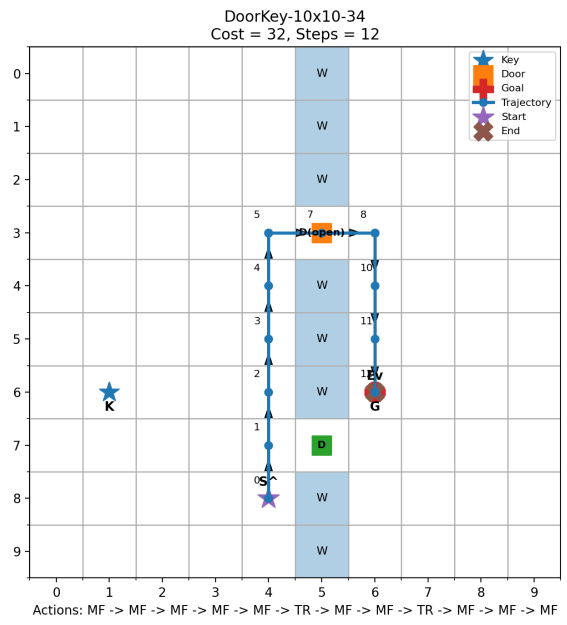


Fig. 41: Trajectory for random environment DoorKey-10x10-34.

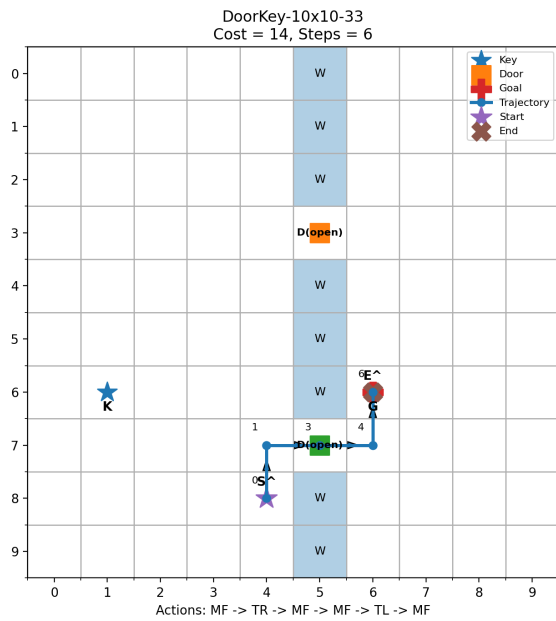


Fig. 40: Trajectory for random environment DoorKey-10x10-33.

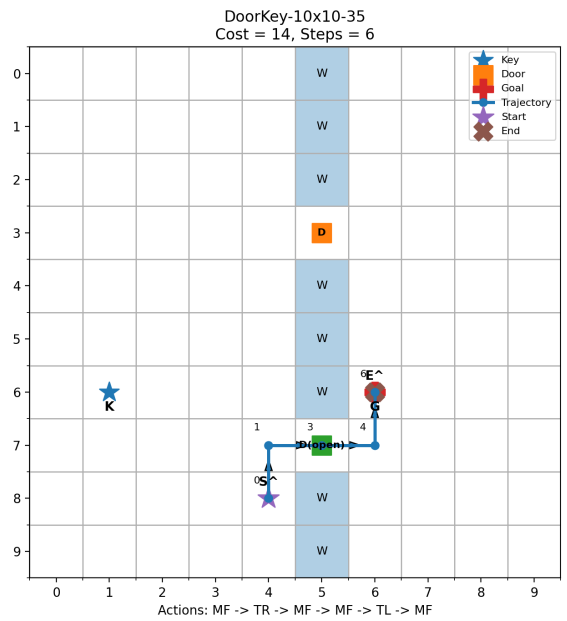


Fig. 42: Trajectory for random environment DoorKey-10x10-35.

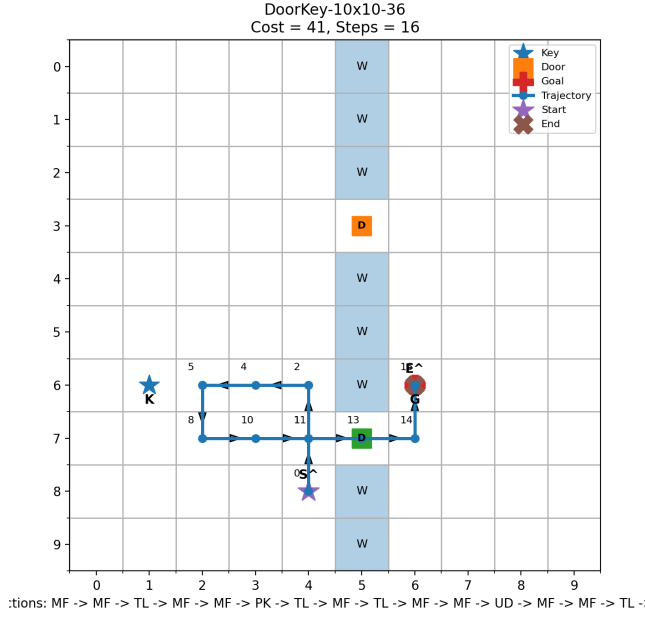


Fig. 43: Trajectory for random environment DoorKey-10x10-36.

C. Discussion of Algorithm Performance

Overall, the dynamic programming framework successfully solved both the known-map and random-map Door-Key environments. The planner was able to consistently generate feasible and low-cost trajectories while satisfying the environment constraints, including obstacle avoidance, key pickup, and door unlocking.

1) *What Worked Well:* The Bellman-based dynamic programming formulation worked effectively because the Door-Key problem has a finite and deterministic state space. By explicitly modeling the agent position, orientation, key possession, and door states, the planner was able to compute globally optimal decisions rather than relying on greedy local actions.

For Part (A), the planner successfully generated optimal trajectories for all seven known environments. The resulting policies minimized the cumulative action cost while correctly handling the required interaction sequence:

$$\text{navigate} \rightarrow \text{pickup key} \rightarrow \text{unlock door} \rightarrow \text{reach goal}. \quad (33)$$

The non-uniform action costs also produced meaningful behaviors. Since turning actions were cheaper than forward motion, the planner sometimes preferred additional rotations in order to reduce expensive movements. This demonstrated that the planner correctly optimized total cost rather than simply minimizing the number of steps.

For Part (B), the unified policy formulation generalized successfully across all 36 random environments. Instead of computing a separate hard-coded solution for each map, the algorithm used a single state-feedback policy:

$$\pi(x) : \mathcal{X} \rightarrow \mathcal{U}, \quad (34)$$

which allowed the same policy table to handle different key positions, goal locations, and door states.

The precomputed policy also significantly reduced online computation time during execution because action selection only required a table lookup:

$$u_t = \pi(x_t). \quad (35)$$

2) *What Did Not Work Well:* One limitation of the approach is the rapid growth of the state space. The total number of states increases combinatorially with the number of environment variables:

$$|\mathcal{X}| \propto (\text{positions}) \times (\text{directions}) \times (\text{key states}) \times (\text{door states}). \quad (36)$$

Although the state space remained manageable for the provided environments, larger maps or additional objects would significantly increase memory usage and computation time.

Another challenge was handling invalid or unreachable states. Many theoretically possible state combinations are physically impossible due to wall constraints or inconsistent key/door configurations. As a result, additional filtering logic was required during policy construction.

Furthermore, early implementations based purely on shortest-path replanning did not fully satisfy the “single control policy” requirement for Part (B). The final implementation addressed this issue by explicitly constructing a unified policy over the entire reachable state space instead of recomputing trajectories independently for each environment.

3) *Why the Approach Was Effective:* The approach was effective because the Door-Key problem is naturally modeled as a deterministic shortest-path MDP. Dynamic programming guarantees optimality by recursively solving smaller subproblems through the Bellman equation:

$$V^*(x) = \min_{u \in \mathcal{U}} [\ell(x, u) + V^*(f(x, u))]. \quad (37)$$

This recursive structure enabled the planner to reason globally about future costs instead of making locally greedy decisions.

In addition, the explicit state representation allowed the planner to encode environment interactions such as carrying a key and opening doors. This was essential because the optimal action depends not only on the agent position, but also on the environment state and previously completed actions.

V. CONCLUSION

This project formulated Door-Key navigation as a deterministic shortest-path MDP and solved it using dynamic programming. The augmented state representation allows the planner to handle both continuous navigation decisions and discrete interactions such as picking up keys and unlocking doors. The same implementation works for all known maps and random maps without hand-coded paths.